Progress Report – Parallel PCA with MPI

This is a progress report on parallel PCA with MPI. As the final project for CS542, we are implementing a distributed PCA algorithm with MPI in Python. The project is going quite well and the core computational components of the parallel PCA pipeline are complete. Each MPI rank loads data, computes local statistics, takes part in global reductions, and projects its part of the dataset onto the top principal components. The system employs standard MPI collective operations, such as MPI_Allreduce, for the global computation of the mean and aggregation of the covariance, while rank 0 performs the eigen-decomposition and broadcasts results to the other ranks. The code currently runs the whole thing from end to end, and initial tests confirm that the distributed PCA implementation is both correct and scalable.

We also created a complete automated shell script that configures the virtual environment in Python, installs all dependencies, and runs the MPI code with the right interpreter. So it has made the project portable and convenient to run in WSL. Along the way, several issues were fixed, like a race condition when creating an output directory and a mismatch in the interpreter when running MPI in a virtual environment. All these fixes have strengthened the pipeline and ensured minimal different behavior across ranks.

The next major milestone will be the integration of a real data pipeline. So far, tests have involved generating datasets synthetically on each of the ranks within MPI. The original intention was for PCA to be carried out on data downloaded from some external source, and with the parallel framework showing reliability, we expanded the project to support full distributed loading of CSV, NPY, and NPZ files. This is an extension, rather than a change in goals; it is simply a progression from the foray into getting the core PCA code done into operating on meaningful datasets.

To support analysis and verification of our distributed PCA pipeline, we implemented a visualization script that reads the output artifacts generated by mpi_pca.py. The visualizer automatically loads the saved eigenvalues, eigenvectors, metadata, and per-rank projection files, and generates two primary plots: (1) a variance explained curve that displays the cumulative contribution of each principal component to the total variance, and (2) a 2-D or 3-D scatter plot of the projected data using the selected top-k principal components. These visualizations allow us to confirm that the PCA computation produces meaningful structure and that the pipeline is correctly communicating results from multiple MPI ranks.

There were no major blockers except for the early setup ones, which now have all been sorted out. Things in the pipeline include completing the data ingestion pipeline for real data, strong and weak scaling experiments, collection of performance results, and preparation of the final report. We believe the project is on course, and any feedback regarding what would be considered average sizes for datasets or added performance metrics to further solidify the final submission is welcome.